

AUTOMAÇÃO WETLANDS por SISTEMA IoT

Yara Carvalho Gomes

A automação visa em primeiro plano agilizar o processo de coleta de informações do sistema Wetlands. Foi proposto uma solução IoT com o uso de microcontroladores e sensores para monitorar continuamente os reatores. Estes dados serão armazenados localmente em um cartão de memória, do qual pode ser removido pelo operador e inserido em um computador para futuras análises das informações coletadas.

(Caso queira ver direto a parte prática → pular para página X)

Lógica do sistema

A organização geral do sistema consiste na lógica Mestre-Escravo. Os microcontroladores Escravos são responsáveis por coletar informações de tensão e temperatura de cada reator de Wetlands, e posteriormente enviar estes dados coletados ao microcontrolador Mestre, atuante como receptor central do conjunto. Os ESP32 Escravos são identificados com um código de 1 a 9. O Mestre é único, e é responsável por tratar o tráfego de informações recebidas de todos os Escravos, e armazená-las de forma sequencial em um cartão de memória local.

Uma vez que o local de implantação do circuito conta com acesso à rede elétrica, optou-se por não utilizar baterias para alimentar o sistema IoT, evitando assim gastos. Foram utilizadas 3 fontes chaveadas 12V (Volts) e 5A (Amperes) que convertem a tensão alternada da tomada de 110v para a tensão contínua de 12v. Foi adicionado um módulo Step-Down LM2596S na saída da fonte para regular a tensão e garantir alimentação para todo o sistema.

Foi determinada a coleta de dados a cada 5 minutos, resultando em 12 medições por hora, 288 medições por dia de cada reator. Dessa maneira, garante-se alta resolução e precisão das informações adquiridas dos reatores.

Escravos

O diagrama de conexão para todos os Escravos são semelhantes. Para cada reator haverá um sistema único, com o intuito de deixá-lo modular e facilitar a depuração.

O componente principal é o microcontrolador de baixo consumo ESP32-DevKitV1 desenvolvido pela empresa *Espressif Systems*. Dentre algumas de suas especificações, possui: processador Xtensa Dual-Core 32-bit LX6 com clock máximo de 240MHz; memória Flash programável de 4MB e memória RAM de 520 KBytes; possui 25 portas programáveis para entrada e/ou saída de dados (GPIO). conexão Bluetooth no padrão 4.2; conexão WiFi de 2.4GHz, podendo atuar como ponto de acesso, estação ou ponto de acesso+estação [1].

Para medição da temperatura, foi escolhido o sensor DS18B20 fabricado pela empresa *Dallas Semiconductor*, modelo resistente à água. Opera na faixa de -10°C

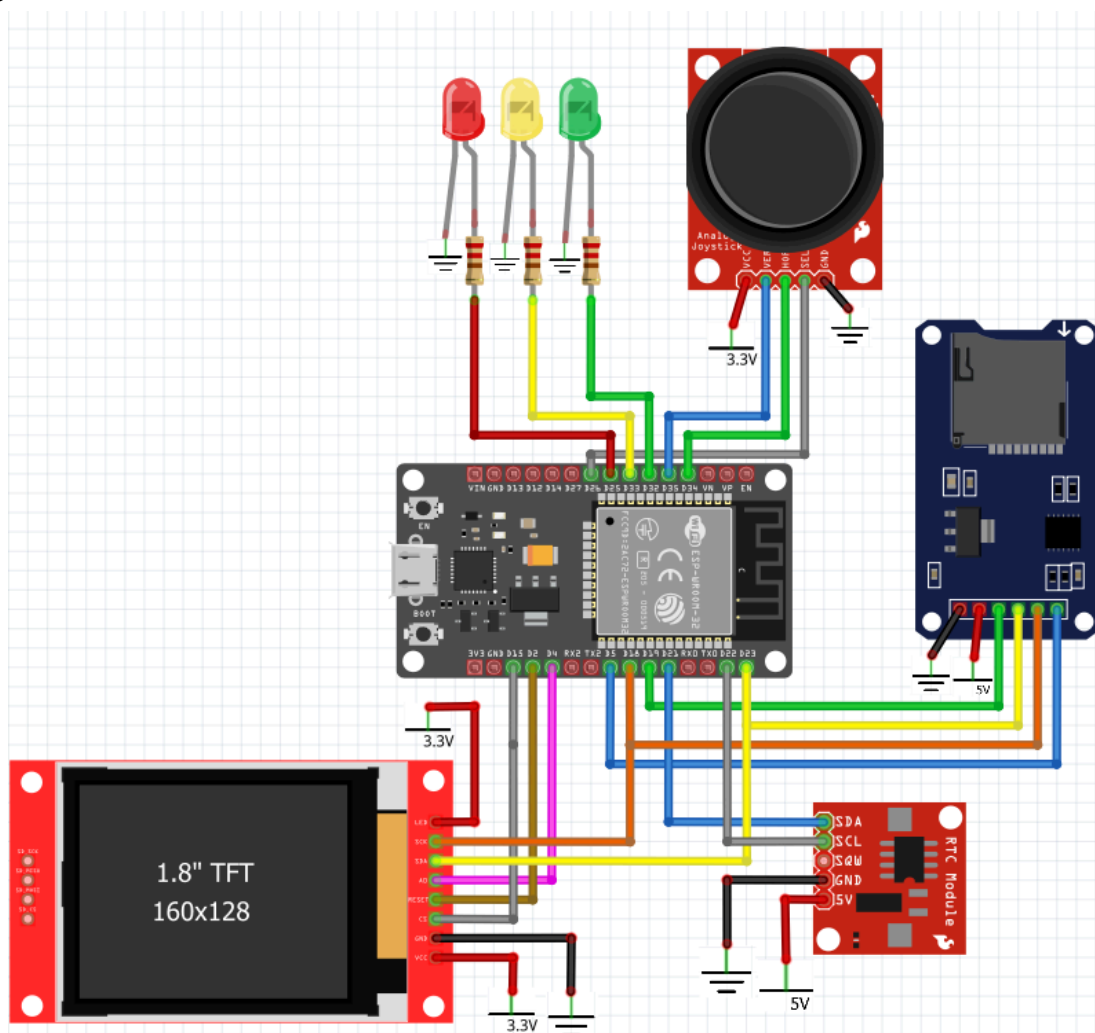
Para salvar os dados advindos dos Escravos, foi utilizado um módulo de cartão de memória microSD genérico. Sua comunicação com o ESP principal é feita

através do protocolo SPI (*Serial Peripheral Interface*)[6]. Os dados são gravados no cartão microSD utilizando o sistema de arquivos FAT32. Isso permite que o arquivo contendo todas as informações seja lido diretamente em qualquer computador compatível.

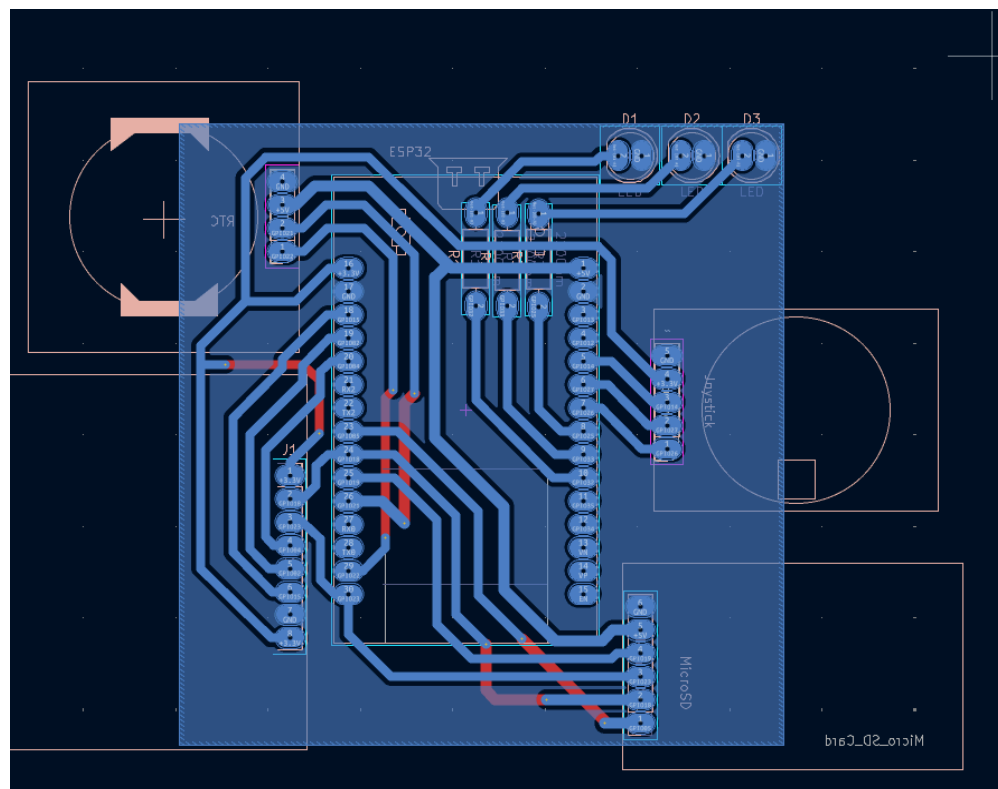
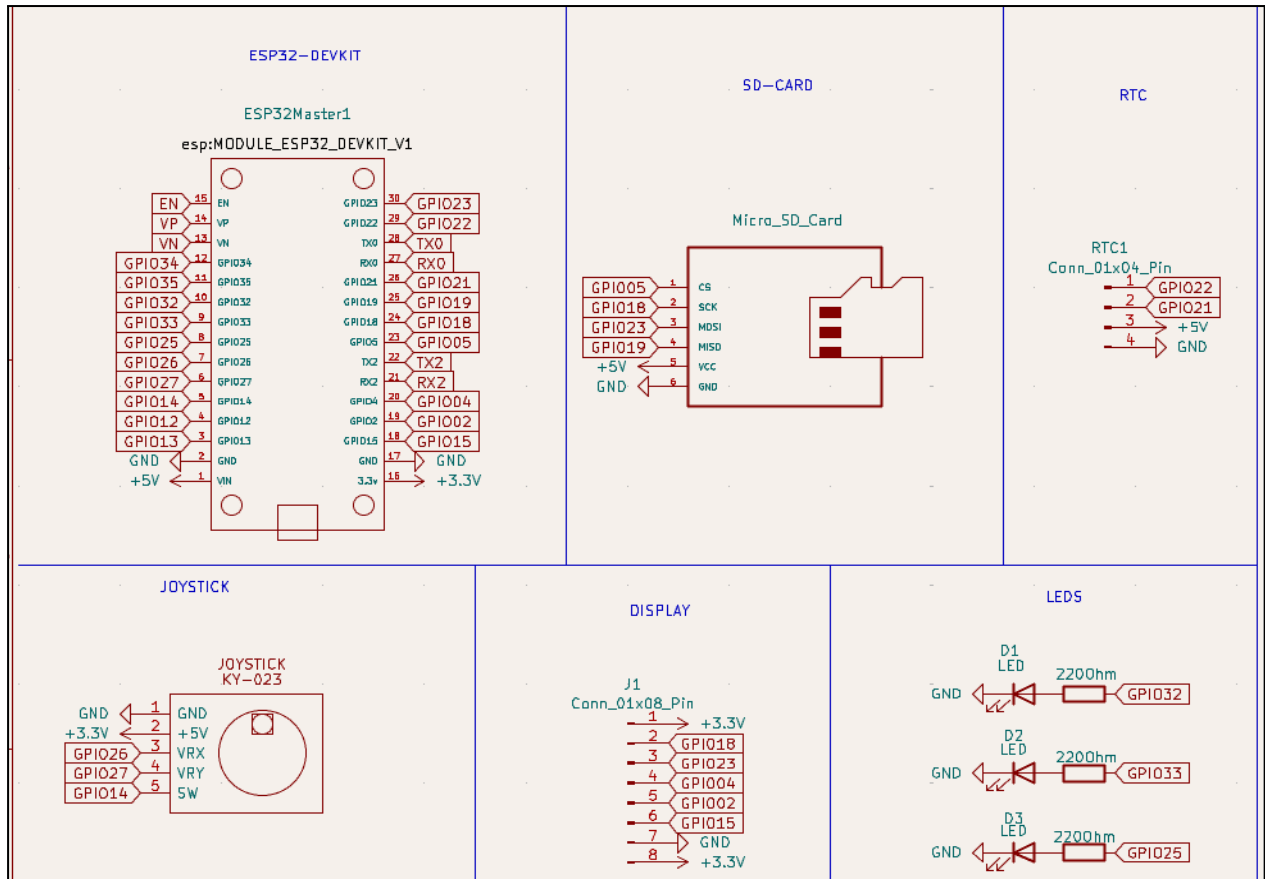
Para facilitar a visualização de algumas informações em campo do sistema, como últimas medições dos reatores e alertas de erro, foi implementado um menu interativo composto por dois módulos. O primeiro, uma tela onde é possível visualizar as informações, o módulo genérico display LCD ST7735 TFT de 1.8 polegadas e que conta com resolução de 128x160 pixels. Sua comunicação com o Mestre é feita via SPI. O próximo, o controle para interagir com o menu, o módulo também genérico KY-023 joystick. Ele possui dois potenciômetros internos responsáveis por identificar a posição de movimento do analógico, além de um botão digital que é ativado quando o joystick é pressionado. Sua conexão com o microprocessador é feita via portas GPIO.

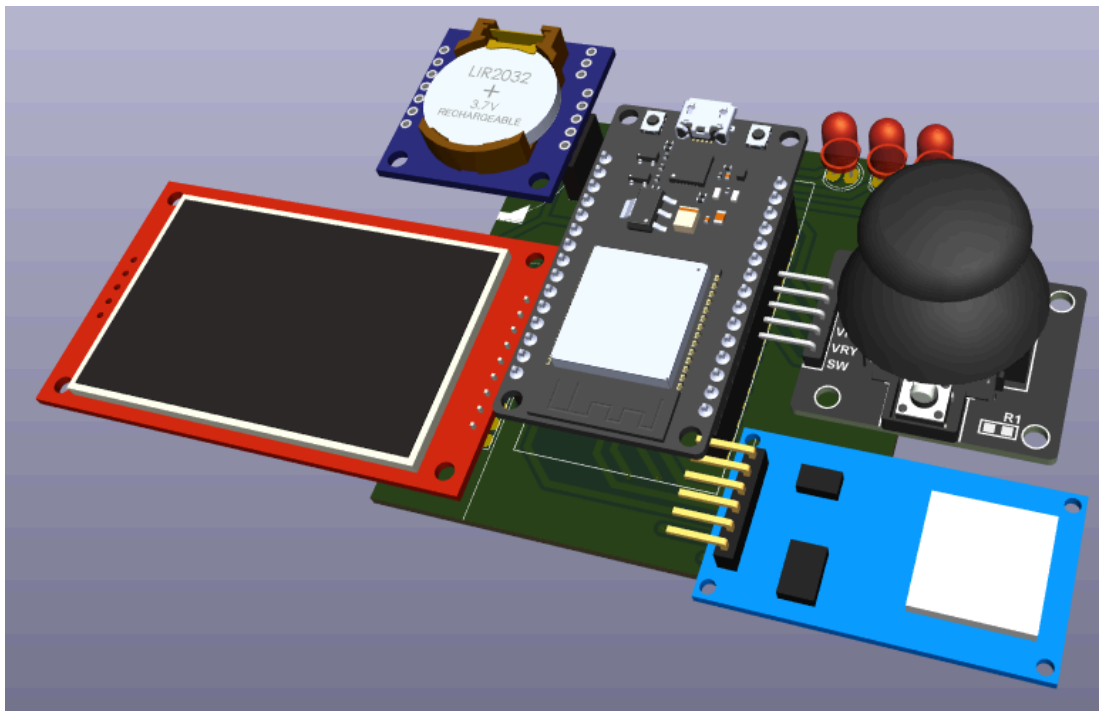
O sistema Mestre será colocado à parte em uma caixa específica e montado sobre uma placa de circuito impressa. O intuito é o operador interagir apenas com o módulo central Mestre, e não com os Escravos.

Segue abaixo o diagrama de conexões do ESP32 Mestre feito no software *Fritzing*.



A placa de circuito impressa (PCB) foi feita utilizando o *software* KiCAD. Segue abaixo o esquema das conexões do sistema, juntamente com o modelo 3D da PCB, sendo uma alternativa para integrar os componentes.





Todavia, com o intuito de deixar ainda mais prático e didático o sistema, foi modelada uma caixa utilizando o software *SolidWorks* para agregar todos os componentes. A caixa foi então impressa utilizando uma impressora 3D modelo *Ender3 V2 Neo*. Modelo 3D da caixa:

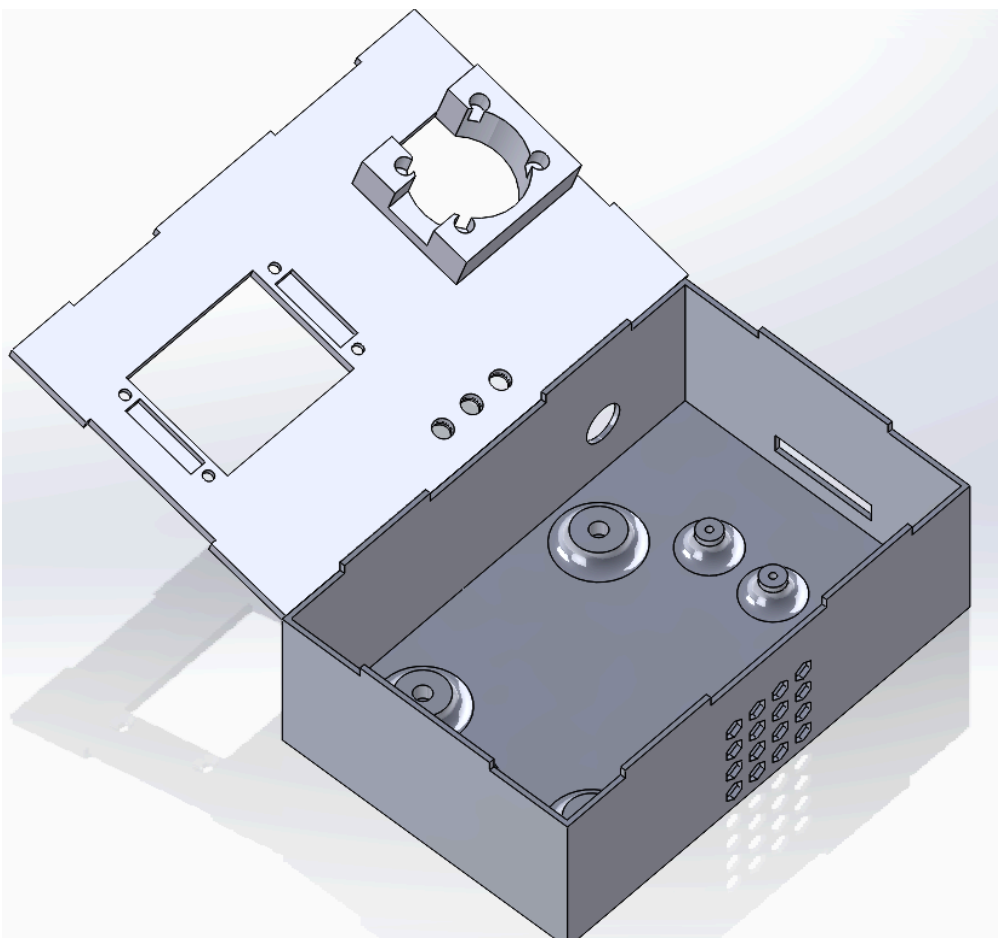


Imagem do sistema real:



Passo a passo de como tudo funciona

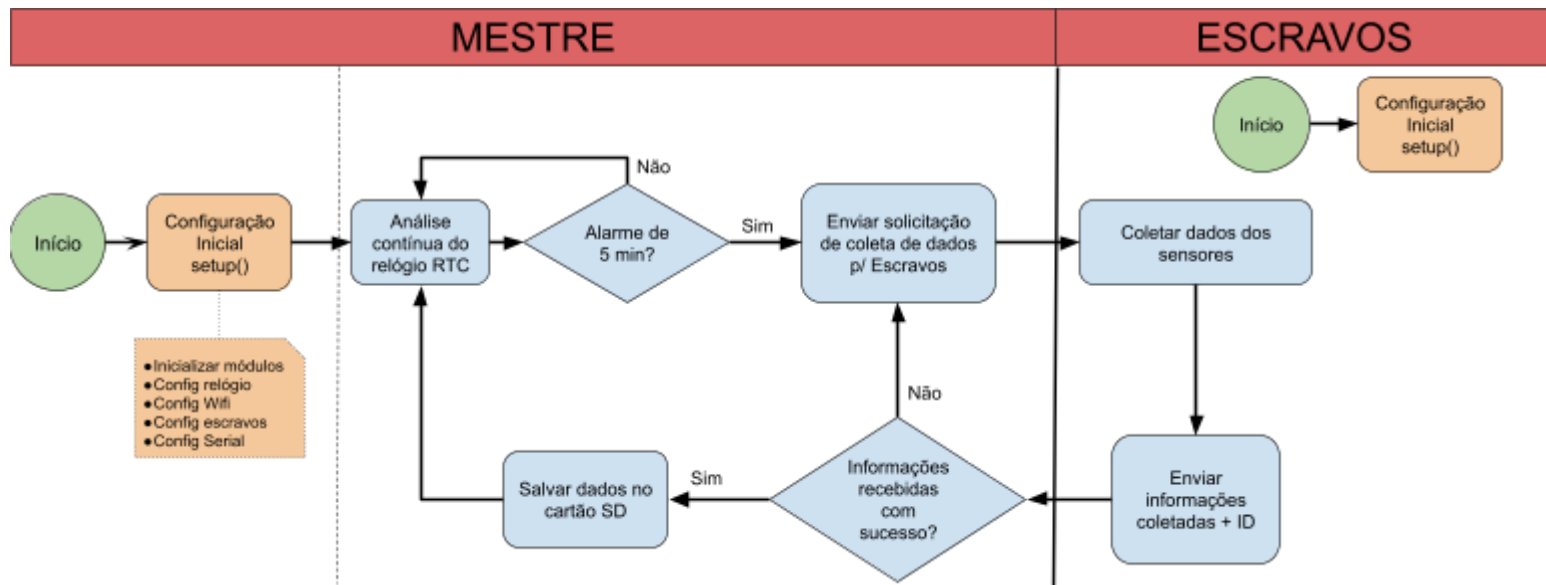
Inicialmente, todo o sistema estará ligado e ativo. O ESP32 Mestre estará continuamente analisando o tempo do sistema baseado no módulo RTC. A cada 5 minutos (tempo estipulado anteriormente) o Mestre enviará um sinal de solicitação para o ESP32 Escravo número 1, para que este faça as medições no tanque em que está acoplado. Após o Escravo 1 coletar os dados por meio dos sensores, ele envia esta informação coletada juntamente com seu identificador (ID) para o Mestre. O Mestre armazena estes dados recebidos (adicionando também o horário de recebimento) de forma sequencial no cartão de memória.

Se o processo acontecer com sucesso no Escravo com ID 1, o Mestre envia uma mensagem de confirmação *Acknowledgment* (ACK) de volta ao ESP32 Escravo a fim de garantir conformidade entre os pares. Em seguida, todo o processo se repete para demais Escravos, continuamente durante todo o dia.

O cartão de memória ficará de fácil acesso em um caixa do qual o operador poderá removê-lo e transferir as informações para um computador, para futuras análises.

Lógica do sistema em forma de diagrama

Diagrama simplificado:



Troca de informação

A troca de informações entre os ESPs é feita via protocolo ESP-NOW. Este protocolo foi desenvolvido pela própria *Espressif* e é uma forma de comunicação sem fio que permite que múltiplos dispositivos se comuniquem entre si sem a necessidade de uma rede Wi-Fi.[7][8]

Mecanismos de Redundância

A fim de evitar perda de dados, alguns mecanismos de segurança foram implementados.

O primeiro mecanismo é referente à coleta de dados dos sensores. O sensor que mede a tensão e o que mede a temperatura, realiza 5 medições nos reatores e pegam a média desses valores como valor final de suas respectivas medições. Dessa maneira garante-se maior estabilidade da informação coletada e evita ruídos acentuados do sistema.

O segundo mecanismo é referente às trocas de mensagem. Para garantir que a mensagem do ESP32 Escravo chegue ao ESP32 Mestre, o Mestre envia de volta ao Escravo uma confirmação *Acknowledgment* (ACK). Caso o Escravo não receba esta confirmação em um intervalo de tempo de 2 segundos, é possível que a informação tenha se perdido ou se corrompido no caminho, e com isso ele a envia novamente, em um total de até 3 vezes.

Ainda no contexto de troca de mensagens, o próximo mecanismo de segurança se encontra no Mestre. Caso a informação recebida de um dos Escravos esteja fora da faixa de valores (Temperatura: $< 0^{\circ}\text{C}$ ou $> 50^{\circ}\text{C}$; $\square$ Voltagem $\leq 0.00\text{V}$) ou ainda que temperatura = -127°C e de tensão = 0.00V , uma mensagem de erro irá piscar no display. Estes valores indicam que algo não está de acordo nas medições, e o display indicará em qual reator se encontra o conflito.

Já com o intuito de evitar possíveis perdas de dados na ausência do cartão de memória durante a transferência de informação, o cronômetro de 5 minutos do Mestre é pausado ao remover o cartão SD. Além disso, um LED vermelho irá prover

informação visual do momento de transferência de dados, indicando que é aconselhável evitar manusear o cartão SD naquele instante. Um LED verde indica que é possível removê-lo sem perigo de perdas. Além disso, após salvar as informações no cartão de memória, o Mestre faz a leitura do último registro gravado para confirmar que a escrita foi realizada corretamente.

Armazenagem e análise de informações coletadas

As informações serão salvas no cartão SD de forma sequencial em um mesmo arquivo. Caso seja o primeiro registro e o arquivo não exista, o Mestre cria este arquivo.

As informações serão salvas seguindo o padrão:
DATA,ID,TEMPERATURA,TENSÃO.

Formato da DATA:
HH:mm:ss:DD:MM:AAAA (hora - minuto - segundo - dia - mês - ano).

Um exemplo de possível registro no cartão SD:
16:15:39:12:10:2025,5,25.71,0.342
“16 horas, 15 minutos, 39 segundos, dia 12, mês 10, ano 2025, ID=5, temp=25.71°C, voltagem=0.342V”

O formato do arquivo gerado é do tipo *Comma Separated Value* (.csv). É um arquivo de texto simples que armazena dados tabulares separados com vírgula, e é compatível com inúmeros programas, inclusive o *Microsoft Excel* e *Google Sheets*. Assim, os dados podem ser facilmente analisados da forma que convier ao pesquisador.

Fontes:

- [1] <https://www.robocore.net/wifi/esp32-wifi-bluetooth>
- [2] <https://www.alldatasheet.com/datasheet-pdf/view/58557/DALLAS/DS18B20.html>
- [3] <https://www.ti.com/lit/ds/symlink/ads1115.pdf?ts=1741234332539>
- [4] https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [5] <https://www.alldatasheet.com/html-pdf/58481/DALLAS/DS1307/180/1/DS1307.html>
- [6] Desenvolvimento de solução IoT para monitoramento de gases
- [7] https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
- [8] <https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>